

Setting Up a Blinking LED Project on STM32F407 Discovery

Initial Information You Need

Before starting, ensure you have:

1. Hardware:

- STM32F407VG Discovery board
- USB Mini-B cable (for programming and power)

2. Software:

- STM32CubeMX (for configuration)
- STM32CubeIDE (for coding and debugging)
- ST-LINK drivers (usually installed with CubeIDE)

3. Board Knowledge:

- The STM32F407 Discovery has 4 user LEDs:
 - **LD3 (Orange)** - PD13
 - **LD4 (Green)** - PD12
 - **LD5 (Red)** - PD14
 - **LD6 (Blue)** - PD15
 - All LEDs are on **GPIOD** pins 12-15
-

Step-by-Step Setup Process

Step 1: Create New Project in STM32CubeMX

1. Launch **STM32CubeMX**
2. Click "**New Project**" or **File** → **New Project**
3. In the MCU/MPU Selector:
 - Type "STM32F407VG" in the search box
 - Select **STM32F407VGTx** (this is the exact chip on Discovery board)
 - Click "**Start Project**"

Why this matters: You must select the exact microcontroller model to get correct pin configurations and peripheral settings.

Step 2: Configure the Clock System (RCC)

1. In the left panel, expand **System Core**
2. Click **RCC** (Reset and Clock Control)
3. Configure:
 - **High Speed Clock (HSE):** Select "**Crystal/Ceramic Resonator**"
 - **Low Speed Clock (LSE):** Leave as "Disable" (not needed for LED blink)

Why this matters: The Discovery board has an 8 MHz external crystal. Using HSE provides more accurate timing than the internal oscillator.

Step 3: Configure Debug Interface (SYS)

1. In the left panel, click **SYS** (System)
2. Set **Debug:** Select "**Serial Wire**"

Why this matters: This enables debugging through the on-board ST-LINK programmer. Without this, you won't be able to program or debug your board.

Step 4: Configure GPIO Pins for LEDs

1. In the **Pinout view** (the chip diagram):
 - Find pin **PD12** → Left-click → Select **GPIO_Output**
 - Find pin **PD13** → Left-click → Select **GPIO_Output**
 - Find pin **PD14** → Left-click → Select **GPIO_Output**
 - Find pin **PD15** → Left-click → Select **GPIO_Output**
2. The pins will turn green, indicating they're configured as outputs
3. **Optional but recommended:** Give them meaningful labels:
 - Click on **PD12** → In Configuration panel, set **User Label:** "LED_Green"

- **PD13:** "LED_Orange"
- **PD14:** "LED_Red"
- **PD15:** "LED_Blue"

Why this matters: GPIO_Output mode allows the microcontroller to drive these pins HIGH or LOW to turn LEDs on/off. Labels make code more readable.

Step 5: Configure Clock Tree

1. Go to **Clock Configuration** tab (top of window)
2. Configure:
 - **Input frequency:** Should show 8 MHz (from HSE)
 - **PLL Source Mux:** Select **HSE**
 - In **HCLK (MHz)** box at the end: Type **168** and press Enter
 - CubeMX will automatically calculate PLL multipliers/dividers

Why this matters: The STM32F407 can run up to 168 MHz. This gives you maximum performance. CubeMX automatically calculates the complex PLL settings needed.

Teaching tip: You can start with a lower frequency (like 16 MHz) for initial learning, then increase it later to show performance differences.

Step 6: Project Settings

1. Go to **Project Manager** tab
2. Configure:
 - **Project Name:** "Blink_LED" (or your choice)
 - **Project Location:** Choose your directory
 - **Toolchain/IDE:** Select **STM32CubeIDE**
 - **Application Structure:** "Basic" is fine for beginners
3. In **Code Generator** tab:
 - Check **"Copy only the necessary library files"** (reduces project size)

- Check **"Generate peripheral initialization as a pair of '.c/.h' files per peripheral"** (better organization)
-

Step 7: Generate Code

1. Click **"Generate Code"** button (gear icon) or press **Alt+K**
2. CubeMX will create all initialization code
3. When prompted, click **"Open Project"** to launch STM32CubeIDE

What happens here: CubeMX generates hundreds of lines of initialization code for clocks, GPIO, and system setup. This saves enormous time!

Step 8: Write the Blinking Code

In **STM32CubeIDE**:

1. In Project Explorer, navigate to:
 - **Core → Src → main.c**
2. Find the while(1) loop in the main() function (around line 90-100)
3. Inside the `/* USER CODE BEGIN 3 */` section, add:

```
/* USER CODE BEGIN 3 */
```

```
HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12); // Toggle Green LED
```

```
HAL_Delay(500); // Delay 500ms
```

```
/* USER CODE END 3 */
```

Complete context:

```
while (1)
```

```
{
```

```
/* USER CODE END WHILE */
```

```
/* USER CODE BEGIN 3 */
```

```
HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12); // Toggle Green LED
```

```
HAL_Delay(500); // Delay 500ms  
}
```

```
/* USER CODE END 3 */
```

Why use USER CODE sections: If you regenerate code from CubeMX, only code outside these sections gets overwritten. Your custom code stays safe!

Step 9: Build the Project

1. Click **Project → Build Project** or press **Ctrl+B**
2. Wait for compilation (should complete with "0 errors")
3. Check the console for "Build Finished" message

Common student errors to watch for:

- Missing semicolons
 - Typos in function names
 - Placing code outside USER CODE sections
-

Step 10: Program and Run

1. Connect your Discovery board via USB cable
2. Click **Run → Debug** (F11) or click the bug icon
3. If prompted about perspective, click "**Switch**"
4. The code will upload and stop at the beginning of main()
5. Click **Resume** (F8) or the play button

Result: The green LED (LD4) should blink every 500ms!

Code Explanation for Students

HAL_GPIO_TogglePin():

- GPIOD - The GPIO port (Port D)

- GPIO_PIN_12 - The specific pin number
- This function flips the pin state: LOW→HIGH or HIGH→LOW

HAL_Delay():

- Blocks program execution for specified milliseconds
 - Uses SysTick timer (configured automatically by CubeMX)
 - Not suitable for production (wastes CPU), but perfect for learning
-

Expanding the Exercise

Once basic blinking works, challenge students to:

1. Blink multiple LEDs:

```
HAL_GPIO_TogglePin(GPIOD, GPIO_PIN_12 | GPIO_PIN_14); // Green and Red
```

2. Create patterns:

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET); // Green ON
```

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_RESET); // Orange OFF
```

3. Running light effect:

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_SET);
```

```
HAL_Delay(200);
```

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_12, GPIO_PIN_RESET);
```

```
HAL_GPIO_WritePin(GPIOD, GPIO_PIN_13, GPIO_PIN_SET);
```

```
HAL_Delay(200);
```

```
// Continue pattern...
```

Troubleshooting Common Issues

No LED blinking:

- Check USB connection
- Verify correct pin numbers (PD12-15)

- Ensure Debug is set to "Serial Wire" in SYS configuration

Cannot program board:

- Install/update ST-LINK drivers
- Try different USB cable or port
- Check that board is powered (ST-LINK LED should be red/green)

Build errors:

- Check for typos
- Ensure code is in USER CODE sections
- Try "Clean Project" then rebuild